

Critical path, workers, and the hybrid schedule

TASK GRAPHS

CRITICAL PATH

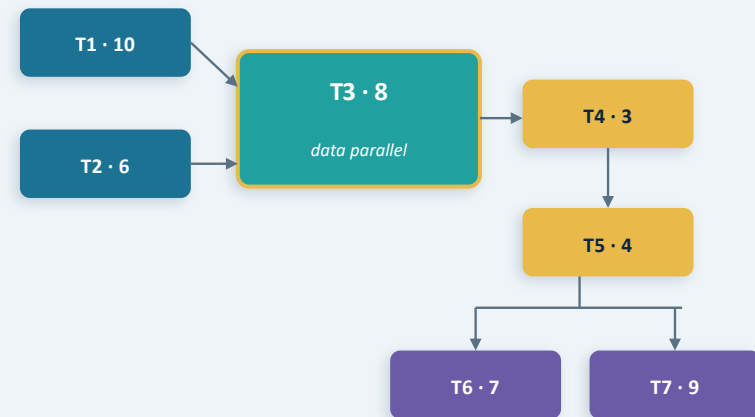
DATA PARALLEL

Givens — task runtimes (minutes)

Task	Description	Time	Depends
T1	Load + partition data	10	—
T2	Build feature schema	6	—
T3	Train (data parallel)	8*	T1, T2
T4	Combine / sync	3	T3
T5	Apply update	4	T4
T6	Evaluate accuracy	7	T5
T7	Evaluate fairness	9	T5

*8 min is the total compute. T3 has 1-min setup + parallel compute + 1-min combine.

Dependency graph



Numbers next to each task = runtime in minutes.

- (a) Max concurrency? How many workers should you provision? (b) Makespan with 1 worker? (c) Makespan with 4 workers? (d) Makespan with 2 workers — and where does the difference come from?

SOLUTION 1

Critical path, workers, and the hybrid schedule

(a) Max concurrency = 4

Widest level of the DAG is T3's data-parallel stage (4 shards in flight). Every other level has ≤ 2 ready tasks. Provision 4 workers — a 5th sits idle.

(b) 1 worker — pure serial

Sum of all runtimes: $10 + 6 + 8 + 3 + 4 + 7 + 9 =$ **47 min**

(c) 4 workers

- $T1 \parallel T2 \rightarrow T2$ (6) hides under $T1$ (10); $T3$ starts at $t = 10$.
- $T3 = 1$ (setup) + $8/4$ (compute) + 1 (combine) = 4 min
- $T6 \parallel T7 \rightarrow T7$ (9) is the long pole; $T6$ finishes 2 min early.
- Path: $10 + 4 + 3 + 4 + 9 = 30$ min

(d) 2 workers — compute phase grows

- $T1$ (10) and $T2$ (6) still overlap. $T3$ starts at 10.
- $T3 = 1 + 8/2 + 1 = 6$ min (was 4 min with 4 workers)
- Then $T4$ (3), $T5$ (4), then $T6 \parallel T7 \rightarrow T7$ (9) on critical path.
- Path: $10 + 6 + 3 + 4 + 9 = 32$ min, i.e. 2 min slower than (c).
- *The whole loss comes from T3: compute = 4 instead of 2. Setup + combine unchanged.*

30

4 workers (c)

32

2 workers (d)

47

1 worker (b)

4

max concurrency (a)

Doubling workers from 2 $\rightarrow$ 4 only saved 2 min, not 3. That gap is the data-parallel serial fraction — see Problem 2.

Amdahl's Law on the data-parallel stage

AMDAHL

SERIAL FRACTION

SPEEDUP CEILING

Setup

Take T3 from Problem 1 in isolation: 1-minute setup, 8 minutes of parallelizable compute, 1-minute combine. On one worker, T3 takes 10 minutes total.

Amdahl's Law

$$\text{Speedup}(N) = 1 / (s + (1 - s)/N)$$

- s = serial fraction (setup + combine, not speedable)
- N = number of workers in the parallel stage
- As $N \rightarrow \infty$, speedup $\rightarrow 1/s$ (a hard ceiling)

Compute

- (a) What is the serial fraction s for T3?
- (b) Speedup at $N = 2, 4, 8, 16$? Fill a table.
- (c) Asymptotic speedup as $N \rightarrow \infty$? Floor on T3 runtime?
- (d) Worth upgrading 4 $\rightarrow$ 16 workers at 4 $\times$ cost, for T3 alone?

Hint: notice that $1 + 8/N + 1$ must match the formula's prediction for every N . Compute both ways as a cross-check.

SOLUTION 2

Amdahl's Law on the data-parallel stage

(a) Serial fraction

Total = 1 + 8 + 1 = 10 min. Serial part = setup + combine = 2 min.

$$s = 2 / 10 = 0.2$$

(c) Asymptote

$\text{speedup}(\infty) = 1 / s = 1 / 0.2 = 5\times$. T3 runtime $\rightarrow$ 2 min (setup + combine), never less.

(b) Speedup table

N	0.8/N	s + 0.8/N	Speedup	Runtime
1	0.800	1.000	1.00×	10.0 min
2	0.400	0.600	1.67×	6.0 min
4	0.200	0.400	2.50×	4.0 min
8	0.100	0.300	3.33×	3.0 min
16	0.050	0.250	4.00×	2.5 min

(d) 4 $\rightarrow$ 16 workers at 4× cost, T3 alone

- Speedup gain: $4.00\times / 2.50\times = 1.60\times$ (not 4×)
- Cost multiplier: 4×. Trade: pay 4× to go 1.6× faster — a bad deal on T3 alone.
- Time saved per run: $4.0 - 2.5 = 1.5$ min. Worth it only if other stages benefit, or 1.5 min $\times$ thousands of runs starts to matter.
- *Lesson: past the knee, extra workers buy almost nothing. Reduce s instead.*

To go faster than 5×, you must reduce the serial part — not add machines.

Training cost and the spot-instance trade

TCO

SPOT INSTANCES

CHECKPOINTING

Givens — a model trained 3× per week

Hardware	8 GPUs (one node)
Wall-clock per run	12 hours
On-demand rate	\$3.06 / GPU-hour
Spot discount	75% (i.e. \$0.765 / GPU-hr)
Reclaim frequency	≈ once every 8 hours
Checkpoints	every 2 hr, 5 min to write
Restart overhead	≈ 10 min to reload

Compute

- (a) On-demand: cost per run? per year (52 weeks)?
- (b) On spot: expected wall-clock per run, including reclaims?
- (c) On spot: expected cost per run? annual savings vs on-demand?
- (d) Name one situation where spot is the wrong choice.

$$\text{\$} = \text{workers} \times \text{hours per worker} \times \text{\$/hour for that instance}$$

SOLUTION 3

Training cost and the spot-instance trade

(a) On-demand cost

Per run: $8 \times 12 \times \$3.06 = \293.76

Per year: $3 \times 52 \times \$293.76$

$\approx \$45,827 / \text{year}$

(b) Wall-clock with reclaims

- Expected reclaims in 12 hrs: $12 / 8 = 1.5$
- Lost work / reclaim: $\approx 1 \text{ hr}$ ($\frac{1}{2}$ of 2-hr window)
- Restart overhead: $\approx 10 \text{ min} \Rightarrow \text{per reclaim} \approx 1.17 \text{ hr}$
- **Expected wall-clock** = $12 + 1.5 \times 1.17 \approx 13.75 \text{ hr}$

(c) Cost on spot, annual savings

Per run: $8 \times 13.75 \times \$0.765 \approx \84.15

Saved per run: $\$293.76 - \$84.15 = \$209.61$

Annual saved: $3 \times 52 \times \$209.61$

$\approx \$32,700 / \text{year}$ ($\sim 71\%$ cheaper)

(d) When NOT to use spot

- Hard deadline — worst-case reclaim chains stretch wall-clock.
- Not checkpointable, or expensive to restart (long warm-up).
- Job is on the critical path of a larger pipeline.
- Non-idempotent side effects (writes that cannot be replayed).
- **Spot is worth it when: checkpointable + idempotent + off the critical path.**

A 75% price cut easily covers a 15% time penalty — the math almost always favors spot, when the conditions hold.

Sizing the serving tier

QPS

REPLICAS

AUTOSCALING

CACHING

Givens — fraud-detection serving API

Peak traffic	2,500 QPS
Per-replica capacity	80 QPS (at p95 = 90 ms)
SLA	p95 ≤ 120 ms
Headroom	30% (above base sizing)
Replica cost	\$ 0.85 / hour
Diurnal pattern	10 hr/day peak; 14 hr/day at 500 QPS

Compute

- (a) Replicas needed at peak (with 30% headroom)?
- (b) Monthly cost at 24×7 fixed sizing?
- (c) If autoscaling, what % compute saved daily vs fixed fleet?
- (d) A cache that hits 35% of requests — replicas needed now?

$$\text{replicas} = \lceil \text{target_QPS} / \text{per_replica_QPS} \rceil \times \text{headroom}$$

Sizing the serving tier

(a) Replicas at peak

Base: $\lceil 2,500 / 80 \rceil = 32$

With 30% headroom: $\lceil 32 \times 1.30 \rceil = 42$

42 replicas

(b) Fixed monthly cost

$42 \times \$0.85 \times 24 \times 30$

$= \$ 25,704 / \text{month}$

\$ 25.7 k / month

(c) Autoscaling savings

- Off-peak replicas: $\lceil 500/80 \rceil \times 1.30 = 10$
- Fixed: $42 \times 24 = 1,008$ replica-hours / day
- Autoscaled: $(42 \times 10) + (10 \times 14) = 560$ / day
- Saved: $1 - 560/1008 \approx 44\%$
- Monthly saving $\approx \$ 11,400$

(d) With a 35% cache hit rate

- Effective load: $2,500 \times (1 - 0.35) = 1,625$ QPS
- Base: $\lceil 1,625 / 80 \rceil = 21$
- With headroom: $\lceil 21 \times 1.30 \rceil = 28$ replicas
- Savings: $42 - 28 = 14$ replicas ($\sim \$ 8,570 / \text{month}$)
- *Plus a latency win — cached responses are sub-ms.*

Three knobs in order: cache → autoscale → replicas. Each cuts the next step's bill before the next step pays for it.

Retraining cadence and total cost of ownership

TCO

RETRAINING

DRIFT

COST OPTIMIZATION

Annual TCO at current (monthly) cadence

Line item	Annual cost
Training compute	\$ 4,800
Serving compute	\$ 42,000
Data + features	\$ 18,000
Engineer time	\$ 60,000
Monitoring + ops	\$ 6,000
Total	\$ 130,800

Drift: 0.4 pp / week. Tolerance: 2 pp before mandatory retrain.

Compute

- (a) Training compute as % of total? Engineer time as %?
- (b) Max retraining interval that keeps the model in SLO?
- (c) New training spend & TCO at that interval?
- (d) Vendor: halve retrains again for + \$12k/yr tooling. Is it worth it on compute alone — and what else might justify it?

Hint: optimizing the smallest line item barely moves the needle. Aim where the dollars actually live.

Retraining cadence and total cost of ownership

(a) Shares of total (\$130,800)

Training compute: $4,800 / 130,800 = 3.7\%$

Engineer time: $60,000 / 130,800 = 45.9\%$

Engineer-time dominates — exactly as the TCO slide predicts.

(b) Max retraining interval

$2 \text{ pp} / 0.4 \text{ pp per week} = 5 \text{ weeks}$

Monthly (~4.3 weeks) is inside SLO but more frequent than necessary — defensive spending.

(c) New training spend at 5-week cadence

Per-run cost: $\$4,800 / 12 = \400

New runs / yr: $52 / 5 = 10.4 \rightarrow 10$

New TCO: $\$130,800 - \$800 = \$130,000$

(d) Vendor tool — \$12k for half the retrains

- Compute saved: $(10 - 5) \times \$400 = \$2,000 / \text{yr}$
- Tooling cost: $+\$12,000 / \text{yr}$
- **Net on compute alone: $-\$10,000 \rightarrow \text{no.}$**
- ***But: if it cuts engineer on-call by even 20%, that's $\sim \$12,000$ from the 46% line — deal becomes plausible.***

The TCO lesson

Cost optimization should target the LARGEST line (engineer time), not the most visible one (compute). The vendor pitch becomes attractive only when you frame its savings against the right denominator.